

บทที่ 3

ทฤษฎีและหลักการ

3.1 การวิเคราะห์และออกแบบระบบเชิงวัตถุ (Object-Oriented Analysis And Design)

การวิเคราะห์และออกแบบระบบเชิงวัตถุ คือการกำหนดแนวทางการปฏิบัติ (Process) และสัญลักษณ์ (Notation) ที่ใช้ในการวิเคราะห์และออกแบบซอฟต์แวร์ ซึ่งจะอ้างอิงตามหลักการของวัตถุ (Object) คือการรวมทั้งคุณลักษณะ (Attribute) และหน้าที่การทำงาน (Operation) ไว้ด้วยกัน

มีการกำหนดมาตรฐานขั้นตอนการปฏิบัติในการวิเคราะห์และออกแบบระบบ ซึ่งในมาตรฐานจะต้องมีการนิยามคำศัพท์ต่างๆ เพื่อความเข้าใจที่ตรงกัน สร้างมาตรฐานของโมเดล (Model) และสัญลักษณ์ที่ใช้ กระบวนการวิเคราะห์และออกแบบระบบเชิงวัตถุ ซึ่งจะประกอบไปด้วยขั้นตอนต่างๆ ดังนี้

1. การกำหนดความต้องการของระบบ (Define User Requirements) โดยใช้ยูสเคสเทคนิค (Use-Case Technique)
 - 1.1 การกำหนดยูสเคส (Use-Case)
 - 1.2 การวิเคราะห์และออกแบบยูสเคส
 - 1.3 การจัดทำส่วนติดต่อกับผู้ใช้แบบกราฟฟิก (Graphic User Interface : GUI)
2. การวิเคราะห์ระบบเชิงวัตถุ (Object-Oriented Analysis : OOA)
 - 2.1 การระบุสิ่งที่สนใจที่มีภายในระบบ (Key Abstraction)
 - 2.2 การสร้างอะนาไลซิสคลาส (Analysis Class)
 - 2.3 การรวมอะนาไลซิสคลาส
3. การออกแบบระบบเชิงวัตถุ (Object-Oriented Design : OOD)
 - 3.1 การออกแบบส่วนประกอบ (Element) เช่นคลาส,แพคเกจ (Package) ,ส่วนติดต่อ (Interface),ระบบย่อย (Subsystem) เป็นต้น
 - 3.2 การเขียนโปรแกรม
 - 3.3 การทดสอบโปรแกรม
 - 3.4 การติดตั้ง

3.2 Unified Modeling Language (UML)

ในการวิเคราะห์และออกแบบระบบ ถ้าเราวิเคราะห์และออกแบบในเชิงวัตถุ มันก็มีวิธีการที่จะเขียน โมเดล เพื่ออธิบายวัตถุ ต่างๆในระบบอยู่หลายโมเดล โดยที่ UML เป็นการรวบรวมโมเดล ต่างๆที่ใช้ในการอธิบายวัตถุ ในระบบมาไว้ด้วยกัน เพราะฉะนั้นการออกแบบโดยใช้ UML ก็จะมองทุกอย่างเป็นวัตถุทั้งหมดมุมมองตามแนวทางของ UML นี้จะแยกออกเป็น 2 ลักษณะหลัก คือ

1. มุมมองที่ไม่มีการเคลื่อนไหว (Static Views) ก็จะทำให้ด้วยลักษณะมุมมองของระบบที่ไม่มีการเคลื่อนไหวหรือคงที่นั่นเอง คือ ไม่ได้มองถึงการเปลี่ยนสถานะหรือสถานะขององค์ประกอบที่อยู่ภายในระบบนั่นเอง

2. มุมมองที่มีการเปลี่ยนไหว (Dynamic Views) จะกล่าวถึงมุมมองของระบบที่มีการเคลื่อนไหวเปลี่ยนแปลง สถานะหรือสถานะขององค์ประกอบที่อยู่ภายในระบบ

3.2.1 ผู้ใช้ระบบ (Actor)

ผู้ที่ใช้ระบบ คือ สิ่งใดๆก็ตามที่อยู่ภายนอกระบบที่กำลังพัฒนา (External entity) ที่มีการปฏิสัมพันธ์หรือมีส่วนร่วมด้วยยูสเคส (Use Cases) ภายในระบบใดๆ โดยสิ่งดังกล่าวอาจเป็น คน, อุปกรณ์ต่างๆ, ระบบอื่นๆ เป็นต้น สำหรับผู้ที่ใช้ระบบจะใช้สัญลักษณ์เป็นรูปตัวการ์ตูน ตามกฎเกณฑ์ของ UML วัตถุประสงค์

เพื่อนำเสนอสิ่งที่อยู่ภายนอกและมีปฏิสัมพันธ์กับระบบที่กำลังพัฒนา นอกจากนี้ยังมีประโยชน์ที่จะใช้ในการกำหนดระดับความปลอดภัยของระบบ ฯ อีกทางหนึ่งด้วย

3.2.2 ยูสเคส

ยูสเคส คือ เอกสารที่บรรยายถึงลำดับของเหตุการณ์ (Event) ที่ผู้ใช้ (Actor) ปฏิบัติการกระบวนการทำงานหนึ่ง ภายในระบบใด ๆ โดยจริงๆแล้วตัวมันเองก็ไม่ได้บอกถึงความต้องการอย่างเป็นทางการของระบบจริงๆ สะท้อนเหมือนกับความต้องการ (Requirement) หากแต่มันมีลักษณะที่บอกความต้องการอย่างไม่เป็นทางการ แฝงไว้ในลักษณะของการเล่าเรื่องมากกว่า ยูสเคสจะใช้รูปวงรี และจะต้องมีชื่อกำกับเสมอ นอกจากนี้แล้วยังมีสัญลักษณ์ที่เป็นเส้นตรงที่แสดงถึงการสื่อสาร (Communication) ระหว่างกันของผู้ใช้และยูสเคส

วัตถุประสงค์

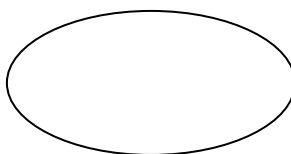
เพื่อนำเสนอมุมมองการโต้ตอบกันระหว่างผู้ใช้กับระบบใดๆ ในการทำงานกระบวนการหนึ่งๆ ภายในระบบใดๆ ลักษณะรายละเอียดจะเป็นแบบมุมมองที่ไม่มีการเคลื่อนไหว

ผู้ที่ใช้ คือ สิ่งที่อยู่ภายนอกระบบที่จะพัฒนาขึ้นทั้งหมด ที่ต้องการแลกเปลี่ยนหรือส่งข้อมูลให้กับระบบ อาจจะเป็นคน ระบบ หรือโปรแกรมอื่น ๆ ก็ได้ โดยมากจะเป็นผู้เริ่มทำงานกับยูสเคสคือสิ่งที่อยู่ภายนอกที่ระบบที่จะพัฒนาขึ้นทั้งหมดที่ต้องการแลกเปลี่ยนหรือส่งข้อมูลให้กับระบบผู้ที่ใช้ อาจจะเป็น คน ระบบ หรือโปรแกรมอื่นๆ ก็ได้ โดยมากจะเป็นผู้เริ่มทำงานกับยูสเคส เช่นผู้ที่ใช้ระบบ เป็นต้น แสดงดังรูปที่ 3-1



รูปที่ 3-1 สัญลักษณ์ของผู้ที่ใช้(Actor)

ยูสเคสจะเป็นตัวแทนงานที่เกิดขึ้นในขั้นตอนต่าง ๆ โดยจะใช้สัญลักษณ์เป็นรูปร่างรี หรือวงกลม แสดงไว้ในรูปที่ 3-2



รูปที่ 3-2 สัญลักษณ์ของงาน

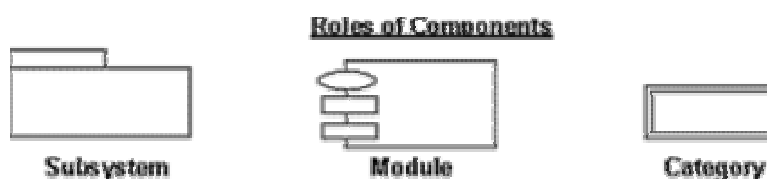
การแสดงความสัมพันธ์หรือการติดต่อสื่อสารกัน(การรับและให้ข้อมูลข่าวสารแก่กันและกัน) ระหว่างผู้ที่ใช้และยูสเคส ซึ่งอาจจะเป็นการสื่อสารแบบทางเดียวหรือ 2 ทางก็ได้ แสดงด้วยสัญลักษณ์เส้น ที่มีหัวลูกศรสีดำ แสดงดังรูปที่ 3-3



รูปที่ 3-3 สัญลักษณ์ของการติดต่อสื่อสารกันระหว่างผู้ที่ใช้กับยูสเคส

3.2.3 ส่วนของการวิเคราะห์

เป็นหลักสำคัญของศึกษาและออกแบบถึงโครงสร้างของระบบงาน ตามขอบเขตของปัญหาที่สนใจมีอะไรบ้าง ตามยูสเคสไดอะแกรมที่ได้จากการหาความต้องการ ในส่วนนี้จะได้ผลลัพธ์เป็นออบเจ็กต์โมเดล (Object Model) ผู้วิเคราะห์จะทำการรวบรวมคลาสที่สัมพันธ์กันขึ้นมาเป็น หมวดหมู่ เรียกว่าคอมโพเน็นซ์ (Component) เพื่อเป็นการลดความซ้ำซ้อนของระบบคอมโพเน็นซ์ไดอะแกรม (Component Diagram) แสดงถึงความสัมพันธ์ระหว่างคอมโพเน็นซ์ต่าง ๆ คอมโพเน็นซ์จำแนกได้ 3 แบบ คือ ระบบย่อย, โมดูล (Module) และ ลำดับชั้น (Category) มีสัญลักษณ์ดังรูปที่ 3-4



รูปที่ 3-4 แสดงสัญลักษณ์ของระบบย่อย, โมดูลและลำดับชั้น

วัตถุต่างๆ ที่มีคุณลักษณะ,หน้าที่การทำงานและความสัมพันธ์(Relationship) ขั้นพื้นฐาน ซึ่งในแต่ละคลาสประกอบไปด้วยรายละเอียดพื้นฐาน 3 ส่วน ดังแสดงในรูปที่ 3-5

Class Name
Attribute
Operation

รูปที่ 3-5 แสดงวัตถุประกอบด้วยคลาส

คำอธิบายส่วนประกอบต่าง ๆ ภายในคลาส

Class Name : แสดงชื่อของคลาสที่กำหนดในระบบ

Attribute : เป็นการกำหนดคุณลักษณะทั้งหมดที่มีภายในคลาส บอกถึงรายละเอียดของข้อมูลภายในคลาสเดียวกันจะไม่ซ้ำกัน แต่ชื่อข้อมูลในคลาสอาจจะไปซ้ำกับชื่อข้อมูลใน คลาสอื่นได้

Operation : เป็นส่วนที่ใช้อธิบายในคลาสนั้นมีวิธีการดำเนินงาน (Method) อะไรบ้าง มีการรับค่าชุดของตัวแปร(Argument) อะไรบ้างหรือมีการส่งค่าออกไปหรือไม่

UML ได้เตรียมความสัมพันธ์ระหว่างคลาสไว้ จะมีความสัมพันธ์อยู่ 4 แบบ มีสัญลักษณ์ดังนี้

1. การเชื่อมต่อกัน (Association) : จะแสดงความสัมพันธ์ระหว่างคลาสจะมีได้ทั้งทางเดียวและสองทาง

2. กลุ่มรวม (Aggregation) : เป็นรูปแบบพิเศษของการเชื่อมต่อคือเป็นความสัมพันธ์ระหว่างส่วนหลักและ ส่วนย่อยของมัน คือส่วนหลักจะประกอบไปด้วยส่วนย่อยต่าง ๆ ของมัน การคงอยู่ของส่วนย่อยจะต้องขึ้นกับส่วนหลัก หรือจะเรียกความสัมพันธ์แบบนี้ว่า โฮล – พาร์ท (Whole-part) มี 2 แบบ

- ความสัมพันธ์แบบบายแวลิว (By Value)
- ความสัมพันธ์แบบบายรีเฟอเรนซ์ (By Reference)

3. ขึ้นอยู่กับ (Depends on) : เป็นรูปแบบความสัมพันธ์แบบหนึ่งที่ใช้แสดงความสัมพันธ์กันระหว่างคลาส 2 คลาส ในแง่ที่คลาส หนึ่งเรียกใช้บริการของอีกคลาสหนึ่ง กล่าวคือคลาสของผู้ขอบริการ ขึ้นอยู่กับบริการของคลาสของผู้ให้บริการ แต่ไม่มีการขึ้นต่อกันภายในโครงสร้างของคลาส

4. ลักษณะทั่วไป (Generalization) : ความสัมพันธ์แบบนี้ใช้แสดงความสัมพันธ์ระหว่างคลาสดับกับคลาส ในแง่ที่คลาสหนึ่งถ่ายทอดคุณสมบัติและโครงสร้างจากอีกคลาส หนึ่ง เรียกคลาสที่ถูกถ่ายทอดว่า: ซุปเปอร์คลาส (Super Class) และเรียกคลาสที่ทำการถ่ายทอดว่าซับคลาส (Sub Class)

3.2.4 ส่วนของการออกแบบ

การออกแบบระบบจะเริ่มเมื่อข้อมูลในช่วงของการวิเคราะห์เพียงพอที่จะเริ่มทำการปฏิบัติกับระบบแล้ว ในการออกแบบควรพิจารณาถึง

1. ฮาร์ดแวร์ว่าระบบเป็น แบบรวมศูนย์ (Centralized) หรือแบบกระจาย (Distributed) ถ้าเป็นระบบแบบกระจายมีการจัดการกับระบบความปลอดภัยอย่างไร จะใช้อะไรเป็นตัวติดต่อสื่อสารต้องการวัตถุเพิ่มเติมเพื่อช่วยอำนวยความสะดวกในการติดต่อสื่อสารหรือไม่ เป็นต้น

2. ซอฟต์แวร์ว่ามีคลาสพิเศษอะไรบ้างเพื่อสร้างหน้าต่างส่วนที่ติดต่อกับผู้ใช้ (User interface) ต้องการคลาสเพิ่มเติมเพื่อช่วยอำนวยความสะดวกในเรื่องของความสัมพันธ์ระหว่างคลาสหรือไม่ถ้าในระบบ

3. ต้องการใช้ระบบฐานข้อมูล มีคลาสพิเศษที่ต้องใช้ในการติดต่อระหว่าง DBMS หรือไม่

4. ต้องมีการลดความซ้ำซ้อนของข้อมูลหรือไม่

5. ต้องทำการ Normalization หรือไม่

6. การใช้คลาสของบุคคลภายนอกหรือไม่ การติดต่อระหว่างวัตถุเป็นแบบพร้อมกัน (Synchronously) หรือ เกิดขึ้นไม่พร้อมกัน (Asynchronously)

ผลลัพธ์ที่ได้จากการพิจารณาส่วนนี้ คือ

1. คลาสเพิ่มเติม

- คลาสเกี่ยวกับการติดต่อกับผู้ใช้งานแบบกราฟฟิคคือหน้าต่าง ต่างๆ (GUI Classes)

- คลาสที่จัดการและติดต่อกับฐานข้อมูล (Database Classes),

2. การสนทนาระหว่างคลาส (Dialogue) และประเภทของวัตถุที่ต้องการเป็นแบบถาวร (Persistent Object)

3. ความสัมพันธ์ระหว่างวัตถุ, การติดต่อ (Transaction) ต่างๆ ที่สนใจ

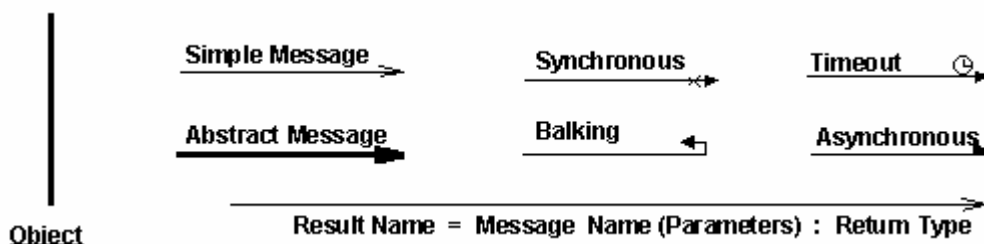
4. โมเดลที่มีการเปลี่ยนแปลง (Dynamic model) จะมีไดอะแกรมแสดงลำดับขั้น (Sequence Diagram) และ ไดอะแกรมแสดงสถานะ (State Diagram)

3.2.5 ไดอะแกรมแสดงลำดับขั้น

คือบทบรรยายรายละเอียดการทำงานภายในยูสเคสใด ๆ ด้วยภาพนั่นเอง ซึ่งจะเป็น ภาพหรือสัญลักษณ์ที่แสดงถึงการโต้ตอบกันระหว่างผู้ใช้งานกับระบบที่กำลังพัฒนา (System) ในลักษณะของการร้องขอบริการของผู้ที่ใช้ระบบ และการตอบสนองของระบบ ตามลำดับของเหตุการณ์ก่อน-หลัง โดยที่ตัวไดอะแกรมแสดงลำดับขั้น เองจะแสดงถึงกลุ่มของวัตถุต่างๆ ที่ต้องทำงานร่วมกัน (Collaboration) เพื่อให้บรรลุเป้าหมายการทำงานของยูสเคสหนึ่งๆ ทั้งนี้การโต้ตอบระหว่างวัตถุหรือผู้ที่ใช้กับวัตถุจะอยู่ในลักษณะของผู้ส่งข่าว (Sender Object) กับผู้รับข่าวสาร (Received Object) ตามลำดับส่วนข่าวสาร (Message) ที่จะส่งหากันนั้นจะใช้เส้นลูกศรหัวสี่เหลี่ยมแสดง และมีการกำหนดชื่อกำกับเส้นลูกศรดังกล่าวด้วยเพื่อแสดงให้เห็นความหมายของข่าวสารดังกล่าวว่าจะให้วัตถุที่รับข่าวสารนั้นทำหรือปฏิบัติอะไร

วัตถุประสงค์

เพื่อแสดงหรือบรรยายการทำงานได้ตอบกันระหว่างผู้ใช้ระบบกับระบบในลักษณะของภาพหรือสัญลักษณ์ ในลักษณะตามลำดับเหตุการณ์ก่อน-หลัง ซึ่งสิ่งเหล่านี้จะมีประโยชน์ในการกำหนดหรือสร้างคลาสของกลุ่ม วัตถุที่มีในไดอะแกรมแสดงลำดับขั้นตอนพฤติกรรม (Behavior) หรือวิธีการดำเนินงานที่กลุ่มของ วัตถุหรือคลาสจำเป็นต้องมีในการทำงานให้บรรลุวัตถุประสงค์ของการสร้างคลาสนั้นขึ้นมาใช้งานภายในระบบที่กำลังพัฒนานั้น



รูปที่ 3-6 แสดงสัญลักษณ์ที่ใช้ในไดอะแกรมแสดงลำดับขั้น

คำอธิบายส่วนประกอบต่าง ๆ ภายในไดอะแกรมแสดงลำดับขั้น

Object : คือ Instance ของคลาสที่ทำหน้าที่รับ-ส่งข้อความ (Message) จากวัตถุอื่นๆ และทำการตอบสนองตามข้อความนั้น ๆ เพื่อให้เกิดการทำงานในขั้นตอนต่าง ๆ ของระบบโดยจะใช้สัญลักษณ์เป็นเส้นตรงแนวตั้งและมีชื่อของวัตถุและคลาส กำหนดอยู่ด้านล่าง

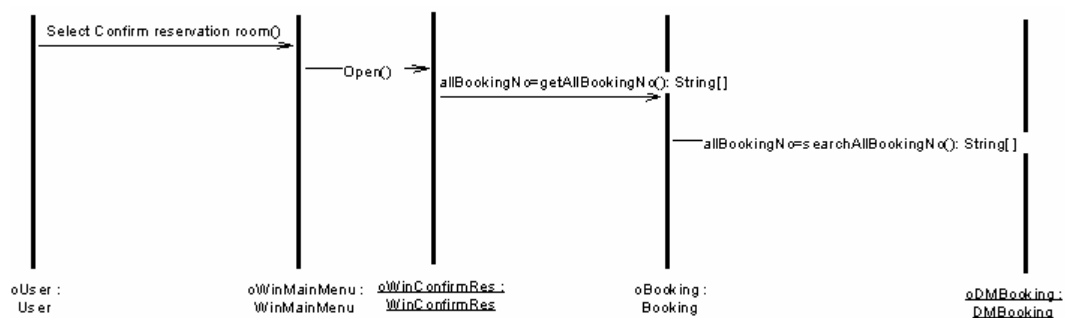
Message : เป็นข้อความที่ส่งไปมาระหว่างวัตถุเพื่อสั่งให้วัตถุทำงานตามที่กำหนด กล่าวได้ว่าการรับ-ส่งข้อความคือชื่อของ วิธีการดำเนินงานของวัตถุที่ถูกเรียก การรับ-ส่งข้อความจะประกอบไปด้วยส่วนประกอบต่างๆ ดังต่อไปนี้

Result Name : คือ ตัวแปรที่จะจัดเก็บค่าที่ได้จากการส่งข้อความ

Parameters : คือ ข้อสรุป (Argument) ต่าง ๆ ที่ส่งให้กับวัตถุเพื่อทำงานตามวิธีการดำเนินงานที่กำหนดด้วยข้อความนั้น ๆ

Return Type : คือประเภทของข้อมูลที่ส่งกลับมากลังกการทำงานของการส่งวิธีการดำเนินงาน นั้น ๆ ถ้าวิธีการดำเนินงานนั้น กำหนดว่ามีการส่งค่ากลับมา

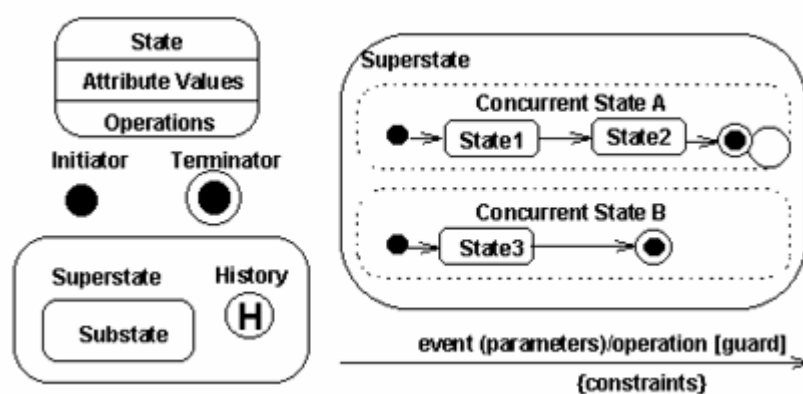
Constraint : คือ เงื่อนไขในการส่งข้อความ



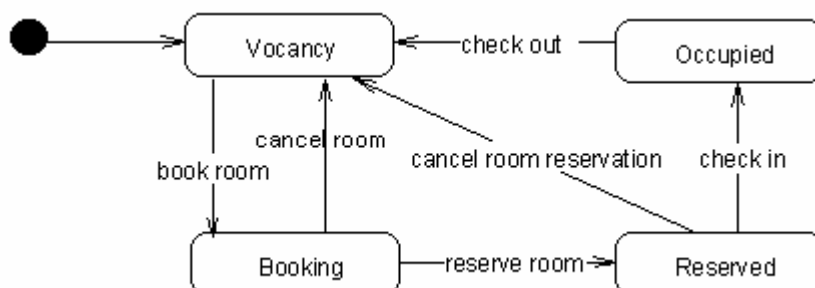
รูปที่ 3-7 แสดงตัวอย่างไดอะแกรมแสดงลำดับขั้น

3.2.6 ไดอะแกรมแสดงสถานะ

ไดอะแกรมแสดงสถานะเป็นไดอะแกรมที่แสดงให้เห็นว่าวัตถุหนึ่ง ๆ สามารถมีสถานะใดได้บ้าง และเกิดเหตุการณ์ใดจึงมีการเปลี่ยนสถานะ



รูปที่ 3-8 แสดงสัญลักษณ์ที่ใช้ในไดอะแกรมแสดงสถานะ



รูปที่ 3-9 แสดงตัวอย่างไดอะแกรมสถานะของวัตถุของห้องพัก

3.2.7 ส่วนของการเขียนโปรแกรมและทดสอบให้ได้เป็นไปตามการวิเคราะห์

โดยจะใช้การโปรแกรมเชิงวัตถุ (Object Oriented Programming) ซึ่งการแปลงจากการวิเคราะห์และออกแบบไปเป็นการโปรแกรมเชิงวัตถุ ส่วนการทดสอบก็เป็นการตรวจสอบคุณภาพของโปรแกรมที่เราทำขึ้นมาโดยวิธีการตรวจสอบวัตถุ (Object Testing)

การแปลงจากการวิเคราะห์และออกแบบ ไปเป็นการโปรแกรมสามารถเชิงวัตถุ ทำได้ไม่ยุ่งยาก เนื่องจากทั้งการวิเคราะห์และออกแบบเชิงวัตถุกับการ โปรแกรมเชิงวัตถุ นั้นสนับสนุนหลักการของ ออบเจกต์อริเอนเทิล (Object-Oriented) ด้วย คือ

1. Encapsulation : การปกป้องข้อมูล
2. Inheritance : การสืบทอดคุณสมบัติ
3. Polymorphism : การที่ได้รับข้อความเดียวแล้วสามารถตอบสนองได้หลายรูปแบบ

ภาษาที่สนับสนุนหลักการของ Object-Oriented

- C++
- Smalltalk
- Eiffel
- Object C
- Object Pascal (หรือ Delphi)
- Java

การวิเคราะห์และออกแบบระบบเชิงวัตถุ	การโปรแกรมเชิงวัตถุ(Java)
Component	เป็น package ในภาษา Java
Class	Class
Attribute	Attribute
Method	Method
Attribute ต่างๆ ถ้าเป็น Read Only	เป็น set method
Attribute ต่างๆ ถ้าเป็น Read-Write	เป็น get, set method
Inherit	Sub class จะต้อง extends จาก Super Class
ถ้า Component มีการอ้างอิงถึง Module หรือ Component อื่นๆ	จะต้อง import package นั้น
access modifier : public (+)	เป็น public ในจาวา
private (-)	เป็น package ในจาวา
protected (#)	เป็น protected ในจาวา
implementation (?)	คือกำหนัด ณ ขณะ Implement เราอาจจะใช้ private ในภาษาจาวาก็ได้แล้วแต่ขอบเขตกรณีที่เราศึกษา

**ตารางที่ 3-1 แสดงการเปรียบเทียบระหว่างการวิเคราะห์และ
ออกแบบระบบเชิงวัตถุกับการโปรแกรมเชิงวัตถุ**

3.2.8 ส่วนของการดูแลและการปรับปรุงระบบ

ในการดูแลนั้นจะทำให้สะดวกกว่า ยกตัวอย่าง เช่น ถ้ามีการเปลี่ยนแปลงโครงสร้างของคลาสใด คลาสหนึ่งก็จะทำการเปลี่ยนแปลงเฉพาะคลาสนั้นเท่านั้นคลาสอื่นๆที่เกี่ยวข้องไม่จำเป็นต้องเปลี่ยนแปลง